Trabajo Práctico Integrador — Segunda entrega (Unidades 4 a 8)

Base de Datos II · Tecnicatura Universitaria en Programación · UTN Facultad Regional Mendoza · Comisión 4
Profesor: Neira Sergio · Integrantes: Ávila Lucas, Liendo Mateo y Pagano Amanda
Fecha: 12 de noviembre de 2026

Repositorio de la entrega: https://github.com/ALucrecia-Pagano/Base_de_datos2

Las unidades 7 y 8 se trabajaron primero en un repositorio aparte (https://github.com/ALucrecia-Pagano/Bases-de-datos-II-NoSQL). Siguiendo la indicación del profesor, lo incorporamos a este repositorio en la rama `unidades-7-8`, conservando sus 38 commits, y después lo integramos a `main` (pull request #4). Así la capa de datos completa queda en un solo repositorio y en una sola rama. Comprobamos que la unificación no alteró el contenido: 286 de los 287 archivos de la parte relacional y 81 de los 90 del repositorio NoSQL quedaron idénticos byte a byte. Los que cambiaron son los archivos de configuración que compartían los dos repositorios (`.gitignore`, `.gitattributes`, README, AGENTS y CONTRIBUTING) y cuatro documentos en los que solo se corrigieron rutas por el cambio de carpeta.

Organización del repositorio

Carpeta	Unidades	Contenido
Relacional_PostgreSQL	1 a 6	PostgreSQL: los TPs de la primera entrega y, para esta, TP6 (Unidad 4), TP7 (Unidad 5) y TP8 de mantenimiento (Unidad 6)
NoSQL_y_Movil	7 y 8	MongoDB (`TP1_FoodStore_NoSQL`) y SQLite con sincronización simulada (`TP8_Unidad8`)

Cada carpeta tiene su README y sus instrucciones para asistentes de IA. El informe sigue los cinco puntos de la consigna: en cada objetivo contamos qué implementamos, cómo lo probamos, qué resultados obtuvimos y qué decidimos. El uso de IA está en su propio apartado y al final hay una matriz con la evidencia de cada objetivo.

Unidad 4 — Normalización avanzada y redundancia

Objetivo 1. Refactorización hasta FNBC

Partimos de la relación heredada `control_lote_almacen(lote, depósito, responsable)`, con las dependencias del dominio LD → C (un lote en un depósito tiene un único responsable) y C → D (cada responsable pertenece a un solo depósito). Calculamos las clausuras, encontramos las claves candidatas LD y LC y comprobamos que la relación está en 3FN pero no en FNBC, porque C determina D sin ser superclave. Eso generaba anomalías concretas de inserción, borrado y actualización, que describimos con ejemplos.

La descompusimos en `responsable_deposito(C, D)` y `control_lote(L, C)`. La reunión por C es sin pérdida porque C es clave de la primera tabla. Dejamos escrito que LD → C ya no se garantiza con las restricciones locales, con un contraejemplo.

En `foodstore_copia_trabajo`, dentro de una transacción, el script migró los datos y una vista reconstruyó la relación original. El `EXCEPT` en los dos sentidos dio 0 filas y los conteos coincidieron (3 originales, 2 y 3 en las tablas nuevas, 3 reconstruidas). El script termina en `ROLLBACK`.

Archivos: informe_parte1_fnbc.md, tp_fnbc_control_lote.sql, evidencia.

Objetivo 2. Migración sin pérdida, especificada y con plan de reversión

Para esta entrega especificamos la migración en tres archivos (requisitos con criterios de aceptación, diseño y tareas) y escribimos un plan de reversión con cuatro escenarios: falla antes del COMMIT, vuelta atrás con la tabla original todavía presente, vuelta atrás después de haberla retirado y, como último recurso, restauración del respaldo.

Para probar el plan armamos un ensayo de ida y vuelta. Hace la migración completa, retira la tabla original como en un corte real, la reconstruye desde las tablas nuevas con el mismo DDL (misma clave primaria y mismas tres claves foráneas) y la compara con una copia tomada antes de empezar. Antes de reconstruir valida que los datos nuevos respeten LD → C, porque la descomposición no la garantiza y la clave primaria original fallaría.

Lo ejecutamos el 10/11 en `foodstore_copia_trabajo`, con un respaldo previo de la base guardado fuera del repositorio. Terminó con código de salida 0: los cinco avisos de control aparecieron en orden, la comparación contra la copia dio 0 filas, se restituyeron las 3 filas originales y todo cerró con `ROLLBACK`.

Una limitación que dejamos registrada: la validación de LD → C funcionó en el caso válido, pero no la forzamos con un caso que la haga fallar.

Archivos: spec y plan, ensayo_migracion_reversion.sql, evidencia.

Objetivo 3. Desnormalización controlada

El reporte que elegimos fue el top diario de ventas por categoría, que en la base de carga masiva (200.005 pedidos y 499.571 detalles) obligaba a recorrer pedidos y productos para cada detalle. Copiamos en `detalle_pedido` la fecha del pedido, la categoría del producto y el estado de borrado del pedido, y agregamos un índice sobre la fecha copiada.

Lo decidimos porque es una consulta de lectura frecuente y las columnas copiadas cambian poco. Las salvaguardas son tres triggers que mantienen las copias sincronizadas cuando cambia el dato original, una auditoría que compara cada copia con su fuente y la comparación del resultado del reporte antes y después.

Medimos dos rondas con los filtros de borrado lógico: la consulta normalizada promedió 48,42 ms y la desnormalizada 1,65 ms, una reducción del 96,59 %. Los accesos a buffers bajaron de 10.942 a 565. Las dos versiones devolvieron el mismo resultado (Bebidas 5.589.534,40 y Pizzas 5.215.387,22) y la auditoría dio 0 diferencias. Todo se hizo dentro de una transacción con `ROLLBACK`.

No probamos concurrencia entre sesiones ni medimos el costo extra en las escrituras.

Archivos: informe_final_tp6.md, informe de la parte 2, script.

Unidad 5 — Administración y seguridad

Objetivo 4. Usuarios, roles y permisos con mínimo privilegio

Definimos cinco roles de grupo sin login (`rol_app_lectura`, `rol_app_escritura`, `rol_soporte`, `rol_reportes`, `rol_auditoria`) y dos usuarios que los heredan (`app_web` y `admin_datos`). La aplicación puede leer, insertar y actualizar, pero no borrar, porque las bajas son lógicas. Soporte puede ver los datos de los usuarios menos la contraseña y solo puede tocar la columna `eliminado`. El acceso a contraseñas pasa únicamente por dos funciones `SECURITY DEFINER` (autenticar y resetear) con `search_path` fijo y sin permiso para `PUBLIC`.

Lo especificamos antes de escribir el script y lo verificamos con el listado de permisos efectivos y con pruebas negativas, por ejemplo que soporte no pueda leer `contrasena`. Una revisión posterior encontró cuatro problemas (permisos de catálogo de más para auditoría, logins con contraseña de ejemplo, privilegios por defecto desalineados con la especificación y funciones sin `search_path` fijo). Los corregimos, los probamos y los publicamos.

Archivos: roles.sql, spec de roles, corrección de seguridad.

Objetivo 5. Auditoría activada y simulacro de acceso indebido

Activamos el registro nativo de PostgreSQL: conexiones, desconexiones y todas las sentencias, con un prefijo que identifica hora, proceso, usuario, base y origen. Elegimos `log_statement = 'all'` en lugar de `'mod'` porque el simulacro necesitaba ver también las lecturas. Su costo, que las claves quedan en texto plano en el log, lo dejamos documentado.

El simulacro reprodujo un ataque típico: tres intentos de login fallidos, uno exitoso, una lectura masiva con el rol de soporte y un intento de escalar privilegios (`GRANT admin_datos TO app_web`), que PostgreSQL rechazó con el error 42501. A partir del log anonimizado (209 líneas) reconstruimos la secuencia, se la dimos a una IA para que la analizara y contrastamos su respuesta contra el log. Aceptamos lo que acertó y descartamos dos afirmaciones que el log no sostenía. Para un incidente real, la contención que propusimos es validar primero el origen y la cuenta afectada y, si se confirma un acceso no autorizado, revocar las sesiones y restablecer la credencial. El bloqueo del origen queda sujeto a esa investigación. En el laboratorio no bloqueamos direcciones ni cambiamos credenciales, porque se trataba de una prueba autorizada, con datos sintéticos y con el escalamiento rechazado. Sí conservamos las evidencias y comprobamos que no quedaran membresías indebidas. Esa decisión está escrita en el informe del incidente.

Archivos: informe de auditoría, informe del incidente, simulacro_incidente.sql.

Objetivo 6. Auditoría de permisos asistida por IA y decisión humana

Le pedimos a la IA que auditara `roles.sql` contra la especificación: que buscara permisos técnicos faltantes (uso del esquema, secuencias, vistas, privilegios por defecto) y que, como

ejercicio de la consigna, propusiera permisos más amplios "para simplificar". No encontró faltantes bloqueantes. Propuso tres permisos amplios: `GRANT ALL ON ALL TABLES` a `rol_app_escritura`, `GRANT ALL PRIVILEGES ON ALL TABLES` a `app_web` y `GRANT SELECT ON usuario` a `rol_soporte`.

Rechazamos los tres y no aplicamos ninguno. `ALL` incluye `DELETE` y `TRUNCATE`, y eso habilitaría el borrado físico del historial que las bajas lógicas buscan evitar. Darle a soporte toda la tabla `usuario` le expondría el hash de las contraseñas. En los dos casos, un error de permisos futuro se resuelve con un permiso puntual y autorizado. El prompt, la propuesta y la decisión de cada caso están en la bitácora (filas de la Parte A.4).

Archivos: BITACORA.md, README del TP7, fila A.4.

Unidad 6 — Mantenimiento

Objetivo 7. Plan de mantenimiento completo

El plan define un respaldo diario en formato custom (`pg_dump -Fc`), un chequeo de salud diario, mantenimiento semanal y restauraciones de prueba sobre una base descartable (`foodstore_restore_test`), con metas de RPO ≤ 24 h y RTO ≤ 15 min. Cada parte tiene su script:

- `backup_foodstore.ps1` hace el respaldo, guarda su hash SHA-256 y cuenta las filas de cada tabla para tener contra qué comparar.
- `restore_test.ps1` restaura en la base descartable y compara tabla por tabla.
- `maintenance.sql` actualiza estadísticas y limpia.
- `health_check.sql` verifica la integridad: huérfanos, cantidades y precios inválidos.

En la Unidad 6 el respaldo se corría a mano, y lo habíamos dejado anotado como mejora. Para esta entrega lo programamos en el Programador de tareas de Windows: la tarea `FoodStore_Respaldo_Diario` ejecuta el mismo script todos los días a las 23:00. Para probarla la disparamos desde el Programador y no a mano. Terminó con resultado 0 y estado `PASS`, con un respaldo de 46.877 bytes y los 9 controles de integridad en cero.

Las limitaciones que dejamos anotadas son tres. La tarea corre solo con la sesión iniciada, para no guardar contraseñas en el Programador. Si el servicio de PostgreSQL está apagado a esa hora, el respaldo falla y queda registrado como `FAIL`. Y no hay rotación automática de los respaldos viejos.

Archivos: SPEC_U6.md, respaldo programado, maintenance.sql, health_check.sql.

Objetivo 8. Simulacro de desastre contra reloj

Cada integrante hizo su propio simulacro. Partimos del último respaldo, restauramos en la base descartable midiendo el tiempo y verificamos que los conteos y los 9 controles de integridad coincidieran con el origen.

Integrante	Base	Tiempo de restauración (RTO)	Antigüedad del respaldo	Resultado
Amanda Pagano (24/10)	`foodstore_dev`	2,58 s	100,4 s	`PASS`, conteos iguales
Amanda Pagano (24/10)	`foodstore_tp3_carga` (carga masiva)	9,15 s	50,0 s	`PASS`, 499.571 detalles iguales
Mateo Liendo (26/10)	`foodstore_dev`	1,64 s	195,2 s	`PASS`
Lucas Ávila (29/10)	`foodstore_dev`	0,93 s	1.205,9 s	`PASS`

Todos quedaron muy por debajo de la meta de 15 minutos. Cada informe individual tiene su sección de incidente. En el de Amanda, la primera comparación de la base de carga marcó una diferencia que en realidad era el orden de las claves del JSON. Se corrigió para comparar por valor y el script quedó con esa lección incorporada. Como mejora quedó respaldar también roles y permisos (`pg_dumpall --globals-only`), porque hoy se recuperan los datos pero no los dueños ni los permisos.

Archivos: informes de Amanda, Mateo y Lucas.

Unidad 7 — NoSQL

Objetivo 9. Modelado documental y consultas sobre datos reales

Modelamos los pedidos como documentos con el detalle embebido en un arreglo de líneas, porque un pedido y sus líneas siempre se leen juntos. Del cliente guardamos solo un id y un alias ("Cliente N"), para no llevar datos personales. Elegimos los tipos con cuidado para no perder precisión: ids `Long`, cantidades `Int32`, fechas `Date` e importes `Decimal128`.

Extrajimos 5 pedidos reales (con 7 líneas) de `foodstore_dev` con una consulta de solo lectura y los migramos a `foodstore_unidad7.pedidos`. Después verificamos tipos y valores contra el origen. El archivo extraído conserva su hash SHA-256, que está protegido en el repositorio para que Git no le cambie los finales de línea.

Las consultas pedidas (listar, filtrar por estado, buscar por producto y filtrar por total) se corrieron sobre esos datos en MongoDB Compass y mongosh.

Archivos: modelo_documental.md, migracion_pedidos.js, consultas_mongodb.js, resultado_migracion.md.

Objetivo 10. Comparación documental vs. relacional

Corrimos las mismas consultas en PostgreSQL y en MongoDB y los resultados coincidieron. Por estado `ENTREGADO` aparecieron los 5 pedidos en los dos motores. Por producto, los pedidos 1, 3 y 5, con las mismas cantidades y precios históricos.

Además le pedimos a una IA (Gemini) que comparara los dos motores y contrastamos dos de sus afirmaciones. Que MongoDB "no valida dependencias entre campos" resultó imprecisa:

armamos un validador con `$expr` y MongoDB rechazó con el código 121 los pedidos con totales o subtotales mal calculados. Que PostgreSQL garantiza la integridad con claves foráneas y restricciones se confirmó con una prueba dentro de una transacción.

Elegimos mantener PostgreSQL como fuente de verdad de los pedidos. Las reglas de integridad que necesita Food Store vienen dadas por el motor, mientras que en MongoDB hay que escribirlas y mantenerlas a mano. MongoDB quedaría, si hiciera falta, como copia de lectura. No medimos rendimiento, porque con 5 documentos no tenía sentido.

Archivos: comparacion_resultados.md, contraste_ia.md.

Unidad 8 — Bases móviles e integración

Objetivo 11. Base SQLite con esquema y sincronización especificados

Diseñamos la base de la tablet en SQLite con tablas `STRICT`, claves foráneas activadas, precios en centavos enteros y restricciones `CHECK`. Tiene seis tablas: el catálogo en caché, la instantánea del servidor, los pedidos borrador con sus ítems y una cola de sincronización. La especificación (requisitos, diseño y tareas) está en la carpeta de Kiro. La dejamos declarada como retroactiva, porque la escribimos después del código y la auditamos línea por línea contra los scripts.

Lo probamos con un borrador válido que confirma, otro que se deshace sin dejar filas y dos rechazos esperados (cantidad cero y producto inexistente).

Archivos: schema_sqlite.sql, spec, resultado de las pruebas.

Objetivo 12. Sincronización simulada con conflictos

El servidor exporta una vista con precio, stock y disponibilidad de cada producto, y la tablet la importa como instantánea. Un "semáforo" en SQLite evalúa cada solicitud pendiente contra esa instantánea. La regla de oro es que la tablet nunca cambia el stock ni el precio del servidor.

Probamos los casos con cambios hechos de verdad en una copia del servidor (`foodstore_tp8_feria`):

Caso	Situación	Resultado
C2	Cambió el precio	`CONFLICTO`; al aceptar el precio nuevo, con otra solicitud, `SIMULADO_APROBADO`
C3	Stock insuficiente	`RECHAZADO`
C4	Producto dado de baja	`RECHAZADO`
C5	Todo coincide	`SIMULADO_APROBADO`
C6	Reenvío de la misma solicitud	Rechazado por `UNIQUE` sobre el identificador: no se duplica

Antes y después de los casos verificamos que el servidor no cambió (5 pedidos, 7 detalles y el mismo stock).

Archivos: semaforo.sql, casos_2c.sql, resultado_casos_2c.md.

Objetivo 13. Code review final con IA

Revisamos con IA los 75 scripts de toda la cursada (`.sql`, `.js`, `.ps1` y `.py` de los dos repositorios), en modo de solo lectura y sobre una copia. Salieron 24 hallazgos y decidimos cada uno: 20 aceptados, 1 corregido y 3 descartados con su motivo.

Los aceptados que son de TPs ya calificados quedaron registrados como mejora, sin tocar esos trabajos. El que corregimos (F07) era real: un error al recargar la instantánea dejaba la tablet vacía. Lo comprobamos, lo arreglamos y repetimos los casos. Uno de los aceptados (F05, un pedido entregado que se podía borrar) lo comprobamos en la base dentro de una transacción. Uno de los descartados (F09) lo refutamos también en la base.

Archivo: consigna_3c.md.

Objetivo 14. Capa de datos completa

La capa relacional, la documental y la móvil están en este repositorio, con sus specs, todos los scripts versionados en Git, el plan de mantenimiento con el respaldo programado y el registro del uso de IA en cada carpeta. Los aportes de los tres integrantes quedan en el historial de commits.

Uso de IA

Unidad	Herramientas	Para qué	Qué aceptamos	Qué descartamos o corregimos
4	Antigravity (Gemini), Claude	Spec, plan de reversión y ensayo	El ensayo de ida y vuelta	Un escenario que daba por probado un fallo nunca forzado; redacción técnica imprecisa
5	OpenCode, Kiro, Codex	Spec de roles, auditoría de permisos, análisis del incidente	La auditoría sin faltantes; lo que el log confirmaba	Una tabla `usuario` distinta de la real; los tres permisos amplios; dos afirmaciones sin sustento en el log
6	Codex, Claude Code, Claude, Antigravity	Scripts de respaldo, restauración y programación	Los scripts revisados y probados	Un segundo borrado de la base de prueba que no estaba autorizado: se hizo recién después de autorizarlo expresamente
7	Gemini, Claude	Contraste de afirmaciones sobre los motores	La afirmación sobre PostgreSQL	La de MongoDB, por imprecisa
8	Kiro, Copilot, Claude, Antigravity	Spec, scripts y code review	20 hallazgos aceptados y 1 corregido	3 hallazgos descartados; un criterio erróneo en un requisito

El detalle de prompts y decisiones está en las DUIA y bitácoras de cada carpeta: Unidad 4, Unidades 5 y anteriores, Unidad 6, Unidad 7, Unidad 8 — DUIA y bitácora.

Cómo cumplimos el protocolo de IA sobre datos

- Ningún script generado con IA se ejecutó sin leerlo antes. Las pruebas corrieron en copias (`foodstore_copia_trabajo`, `foodstore_restore_test`, `foodstore_tp8_feria`) o dentro de transacciones con `ROLLBACK`, y con un respaldo previo guardado fuera del repositorio cuando había cambios de estructura.
- No le pasamos a ninguna IA datos personales reales: usamos usuarios sintéticos (`@example.invalid`), clientes con alias, IPs de documentación en los logs y contraseñas de prueba. Las credenciales locales están en `pgpass.conf`, fuera del repositorio.
- Los asistentes no hicieron commits ni se conectaron a las bases sin autorización para cada etapa.

Matriz de cumplimiento

Obj.	Evidencia principal	Estado
1	informe_parte1_fnbc.md	Cumplido
2	plan_reversion.md y evidencia	Cumplido
3	informe_final_tp6.md	Cumplido
4	roles.sql	Cumplido
5	informe_incidente.md	Cumplido
6	BITACORA.md, Parte A.4	Cumplido
7	README_programacion.md y SPEC_U6.md	Cumplido
8	Informes individuales de la Unidad 6	Cumplido
9	resultado_migracion.md	Cumplido
10	comparacion_resultados.md	Cumplido
11	schema_sqlite.sql y spec	Cumplido
12	resultado_casos_2c.md	Cumplido
13	consigna_3c.md	Cumplido
14	Este repositorio	Cumplido

Limitaciones conocidas

- La descomposición FNBC no garantiza LD → C por sí sola, y la validación que lo controla en la reversión no se probó con un caso negativo.
- En la desnormalización no probamos concurrencia entre sesiones ni medimos el costo en escrituras.
- Los respaldos no incluyen roles ni permisos globales.
- El respaldo programado depende de que la sesión esté iniciada y el servicio de PostgreSQL encendido.
- En MongoDB no quedaron validadores permanentes en la colección.
- La spec de la Unidad 8 es retroactiva.

- Las mejoras del code review que tocan TPs ya calificados quedaron registradas, no aplicadas.